Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION TECHNOLOGY

NetWatch: Linux Network Activity and Security Monitoring System



Submitted By: Enrolment No.: 992501230047, Name: Akshara Yadav
Enrolment No.: 992501230049, Name: Shambhavi Singh
Enrolment No.: 992501230050, Name: Labhansh Vashisht
Enrolment No.: 992501230060, Name: Mohd Nauman Ali

Course Name: Unix Programming Lab
Course Code: 24B15CS214
Program:B. Tech. CSE (Cyber Security)
Second Year 3rd Semester

Chapter 1: Introduction

1.1 Background

Modern Linux operating systems continuously communicate over computer networks through dozens to hundreds of concurrent Transmission Control Protocol (TCP) and User Datagram Protocol (UDP) sockets. These sockets support essential system daemons (such as systemd-resolved, sshd, and cron), web services, database backends, application endpoints, and transient administrative scripts. Standard Linux utilities such as ss, netstat, lsof, and ps expose low-level networking information. However, their output is command-oriented, distributed across multiple unrelated interfaces, and requires manual cross-referencing. For instance, determining which user application opened a specific listening socket requires running network inspection tools, searching process tables, and manually tracing socket inodes across the /proc filesystem. NetWatch provides a unified, passive Linux network monitoring tool written in C++17 that correlates network sockets directly to owning processes and monitors system state changes in real time.

1.2 Problem Statement

Identified Challenges: Network visibility and endpoint defense on Linux hosts face several practical challenges:

- **1.** Fragmented Visibility: Network socket state and process attribution are split between network tables (/proc/net/tcp, /proc/net/udp) and process descriptor directories (/proc/[PID]/fd/).
- **2.** Manual Process Correlation: Systems administrators and cyber security students must manually correlate socket inode numbers with process file descriptors to identify rogue services.
- **3.** Lack of Real-Time Change Alerts: Native Linux command-line utilities provide static snapshots rather than continuous delta monitoring. When a backdoor or unauthorized service opens a port, it goes unnoticed unless continuous logging is active.
- **4.** Heavyweight Alternatives: Commercial endpoint detection agents introduce high CPU/memory overhead, complex database dependencies, and Docker requirements, making them unsuitable for lightweight workstations and educational environments.

NetWatch addresses these challenges by offering a single, dependency-free command-line system that captures active sockets, identifies listening ports, maps them directly to running PIDs via /proc, performs real-time snapshot diffing, matches user-defined security rules, and maintains persistent audit logs.

1.3 Social & Environmental Relevance

Social Relevance: Cyber security is an indispensable requirement of modern digital society. Unmonitored background network sockets on Linux workstations and servers expose educational institutions, healthcare systems, and critical infrastructure to unauthorized remote access, reverse shells, and malicious command-and-control communication. NetWatch democratizes endpoint security by providing an accessible, open-source monitoring tool that empowers students, researchers, and system administrators to audit their systems passively without requiring expensive proprietary licenses.

Environmental Relevance: Traditional enterprise monitoring tools consume significant CPU and memory resources, increasing server hardware wear, electrical power consumption, and data center cooling requirements. In contrast, NetWatch is implemented in high-performance C++17 using lightweight system calls and 100ms sleep-sliced polling, resulting in sub-1% CPU utilization and near-zero power draw.

Primary Sustainable Development Goal (SDG): SDG 9: Industry, Innovation and Infrastructure — Fosters resilient, secure digital infrastructure through efficient open-source monitoring.

Secondary Sustainable Development Goal (SDG): SDG 16: Peace, Justice and Strong Institutions — Enhances cybersecurity capabilities to prevent data theft, cyber espionage, and unauthorized intrusion.

1.4 Scope and Boundaries

The functional scope of NetWatch includes passive monitoring of active IPv4 and IPv6 TCP/UDP network connections, extraction of listening sockets, resolution of socket inodes to Linux PIDs and binary names via /proc, state delta tracking via periodic snapshot comparisons, rule-based alerts from config/rules.conf, append-only persistent logging to logs/netwatch.log, and an interactive command-line interface. Boundarily, NetWatch is strictly a passive diagnostic application; it intentionally does not terminate processes or inject firewall rules, avoiding system instability.

Chapter 2: System Design & Methodology

2.1 Proposed Methodology

NetWatch employs a modular, pipeline-based system architecture consisting of seven cooperating functional modules. Data gathering modules feed the real-time event monitor, which passes detected state deltas to the alert engine and the persistent logger, all unified under an interactive terminal interface.

Module Name	Source Files	Primary Responsibility	Key Unix Concepts Used
Mod 1: Active Conns	network_monitor.cpp	Collects & parses active TCP/UDP sockets	ss utility, stringstream parsing
Mod 2: Listening Ports	network_monitor.cpp	Filters bound listening services	State validation, socket classification
Mod 3: Process Mapper	process_mapper.cpp	Resolves socket inode to PID & process name	readlink(), opendir(), readdir(), /proc
Mod 4: Event Monitor	event_monitor.cpp	Takes periodic snapshots & computes deltas	signal(), SIGINT, volatile sig_atomic_t
Mod 5: Alert Engine	alert_manager.cpp	Matches events against config/rules.conf	Rule parsing, severity classification
Mod 6: Persistent Logger	logger.cpp	Appends audit events to logs/netwatch.log	mkdir(0755), std::ios::app file I/O
Mod 7: CLI Interface	main.cpp	Interactive menu & CLI arguments	argc/argv, terminal UI, menu loop

2.2 Tool and Technique Selection

Language & Toolchain: C++17 was selected because it provides zero-overhead access to native POSIX system programming primitives while offering modern, type-safe standard library containers (std::vector, std::map, std::string). Compilation is managed by GNU Make using strict warning flags (-std=c++17 -Wall -Wextra -pedantic -O2).

Unix System Programming Primitives: The project applies core Unix system programming concepts in a dedicated command execution helper (command_runner.cpp):

- • **fork():** fork(): Clones the calling process to create an independent child process for command execution.

- • **execvp():** execvp(): Replaces the child process memory image with the target utility binary (e.g. ss).
- • **pipe():** pipe(): Establishes an anonymous unidirectional IPC byte channel between child and parent.
- • **dup2():** dup2(): Duplicates the write end of the pipe onto STDOUT_FILENO (fd 1), capturing output directly.
- • **read() / close():** read() & close(): Performs low-level I/O; closes unused pipe ends to prevent blocking and file descriptor leaks.
- • **waitpid():** waitpid(): Reaps child process exit status, preventing defunct zombie processes in the kernel table.
- • **readlink():** readlink(): Inspects symbolic links in /proc/[PID]/fd/ to resolve socket: [INODE] targets.
- • **signal():** signal(SIGINT): Intercepts Ctrl+C using volatile sig_atomic_t flags for instant, graceful shutdown.

Application of Object-Oriented Programming (OOP): Object-Oriented Programming (OOP) concepts are integrated cleanly to promote modularity and testability:

- **1.** Encapsulation: AlertManager and Logger encapsulate their internal state (rule maps, known processes, log file handles) and expose controlled public member functions (evaluateEvent(), logEvent(), viewHistoricalLogs()).
- **2.** Data Abstraction: Network socket records and security events are modeled as lightweight, structured abstractions (ConnectionInfo, NetworkSnapshot, NetworkEvent, SecurityAlert) hiding raw string manipulation.
- **3.** Modularity & Single Responsibility: Each functional module operates as an independent subsystem with clear inputs and outputs, ensuring easy debugging and unit testing.

Chapter 3: Implementation & Testing

3.1 Experimental Setup

All development, testing, and performance validation were conducted on a physical Linux workstation with the following specifications:

Configuration Parameter	Specification Details
Operating System	Fedora Linux 44 (Workstation Edition), x86_64 architecture
Linux Kernel Version	7.2.13-200.fc44.x86_64 (SMP PREEMPT_DYNAMIC)
C++ Compiler	g++ (GCC) 16.2.1 20260819 (Red Hat 16.2.1-2)
Build Utility	GNU Make 4.4.1
C++ Standard	ISO/IEC 14882:2017 (C++17)
System Utilities	/usr/bin/ss, /usr/bin/ps, /usr/bin/lsof, /usr/bin/python3
Hardware Platform	8-core x86_64 CPU @ 3.8 GHz, 16 GB DDR4 RAM, NVMe Solid-State Storage

3.2 Results and Test Cases

The system was subjected to rigorous laboratory testing across eight distinct functional scenarios to validate active connection monitoring, listening port detection, /proc socket mapping, real-time event detection, suspicious port rule evaluation, persistent logging, and graceful signal handling.

Test ID	Test Scenario	Input / Stimulus	Observed Behavior & Output	Verdict
Test 1	Active Connections	./netwatch -c	Parsed ss -tunapn; formatted active TCP/UDP sockets with endpoints & state	PASS
Test 2	Listening Ports	./netwatch -l	Filtered sockets; displayed listening ports with bound IP addresses	PASS
Test 3	/proc Inode Mapper	python3 -m http.server 3000 &; ./netwatch -m	Decoded /proc/net/tcp, extracted inode 732309	PASS

Test ID	Test Scenario	Input / Stimulus	Observed Behavior & Output	Verdict
Test 4	New Port Detection	Dynamic listener opened on port 8080	Diff detected new listener; printed [EVENT: NEW LISTENER] Port 8080 (TCP)	PASS
Test 5	Port Closed Detection	Listener on port 8080 terminated	Diff detected removal; printed [EVENT: PORT CLOSED] Port 8080 (TCP)	PASS
Test 6	Suspicious Port Alert	Opened port 4444 (Metasploit rule)	Matched config/rules.conf; triggered bold [ALERT: SUSPICIOUS] on port 4444	PASS
Test 7	Audit Log Persistence	Checked logs/netwatch.log	All events & alerts appended in std::ios::app mode; verified with ./netwatch -h	PASS
Test 8	Graceful Signal Exit	Sent SIGINT (Ctrl+C) during monitoring	Trapped signal; sleep slicing exited loop in 100ms with exit code 0	PASS

3.3 Sample Generated Outputs

Output 1: Sample Module 3 Process-to-Port Mapping Output:

```
PORT    PROTO   LOCAL BIND          STATE      INODE      PID    PROCESS
-------------------------------------------------------------------------------
3000    TCP     0.0.0.0             LISTEN     732309     33     python3
5432    TCP     127.0.0.1           LISTEN     188746     34     python3
-------------------------------------------------------------------------------
Total Sockets Mapped: 2
```

Output 2: Sample Module 5 Alert & Module 6 Persistent Log Entry:

```
[2026-11-09 18:41:59] [EVENT: NEW LISTENER] New listening port detected: 4444 (TCP) by process: python3 (PID: 41)
[2026-11-09 18:41:59] [ALERT: SUSPICIOUS] Unexpected local port detected: 4444 (Metasploit_Default_Listener) | Protocol: TCP | Process: python3 (PID 41)
[2026-11-09 18:41:59] [ALERT: WARNING] New listening service opened on port: 4444 (TCP) by process: python3 (PID 41)

Log Record in logs/netwatch.log:
[2026-11-09 18:41:59] | ALERT_SUSPICIOUS | TCP | 0.0.0.0:4444 | 0.0.0.0:0 | PID:41 | python3 | Unexpected local port detected: 4444 (Metasploit_Default_Listener)
```

Chapter 4: Conclusion & Future Scope

4.1 Summary of Work and Achievements

The NetWatch project successfully implements a comprehensive, lightweight network activity and security monitoring system tailored for Unix system environments. By directly exploiting Linux kernel interfaces (/proc/net/tcp, /proc/net/udp, /proc/[PID]/fd/) and low-level POSIX system calls (fork, execvp, pipe, dup2, readlink), NetWatch achieves complete endpoint socket visibility without relying on heavyweight third-party frameworks. Object-oriented encapsulation in the Alert and Logger classes provides clean abstraction barriers and extensible design. The system proved robust, responsive, and completely stable across all test cases.

4.2 Impact on Society, Sustainability, and Ethical Compliance

NetWatch provides an accessible, zero-cost diagnostic solution that strengthens system defense against unauthorized access, backdoors, and malware exfiltration. From an environmental standpoint, its high computational efficiency guarantees minimal CPU and energy utilization. Ethically, NetWatch operates purely as a passive diagnostic monitor; it does not intercept encrypted packet payloads, spoof traffic, or terminate third-party processes, strictly adhering to academic integrity and computer ethics standards.

4.3 Limitations of the Work and Future Scope

Primary limitations include: (1) Inability of unprivileged users to read file descriptors of root-owned daemons without sudo; and (2) Inability to capture sub-second ephemeral outbound sockets that close before a snapshot refresh cycle. Future enhancements include: (1) Kernel-level event streaming via Linux eBPF (Extended Berkeley Packet Filter) or fanotify; (2) Integration with threat intelligence feeds (e.g. AlienVault OTX) for remote IP reputation scoring; and (3) JSON/CEF log formatting for integration with SIEM platforms.

References

[1] W. R. Stevens and S. A. Rago, Advanced Programming in the UNIX Environment, 3rd ed. Boston, MA: Addison-Wesley, 2013.

[2] M. Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook. San Francisco, CA: No Starch Press, 2010.

[3] Linux Kernel Organization, 'The /proc filesystem,' Linux Kernel Documentation, 2024. [Online]. Available: https://docs.kernel.org/filesystems/proc.html.

[4] B. Stroustrup, The C++ Programming Language, 4th ed. Upper Saddle River, NJ: Addison-Wesley, 2013.

[5] IEEE, 'POSIX.1-2017: Standard for Information Technology—Portable Operating System Interface (POSIX) Base Specifications, Issue 7,' IEEE Std 1003.1-2017, 2018.

Appendix: Team Roles and Contributions

Akshara Yadav (Enrolment No.: 992501230047): Module 1 & 2 design; implementation of connection endpoint parsing and ss command argument formatting.

Shambhavi Singh (Enrolment No.: 992501230049): Module 5 implementation; rule file parser design (config/rules.conf) and alert severity classification.

Labhansh Vashisht (Enrolment No.: 992501230050): System architecture; Module 3 /proc socket inode mapping using readlink(), Unix command helper (fork, pipe, dup2), and Module 4 snapshot diffing.

Mohd Nauman Ali (Enrolment No.: 992501230060): Module 6 persistent file logging, Module 7 interactive command-line interface, Makefile build automation, and test suite execution.